LANGUAGES AND AUTOMATA I

HOMEWORK #2 PARTIAL ANSWERS

Exercises.java

```
import java.util.List;
import java.util.Map;
import java.util.HashMap;
import java.util.Optional;
import java.util.function.Predicate;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Exercises {
    static Map<Integer, Long> change(long amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Amount cannot be negative");
        }
        var counts = new HashMap<Integer, Long>();
        for (var denomination : List.of(25, 10, 5, 1)) {
            counts.put(denomination, amount / denomination);
            amount %= denomination;
        }
        return counts;
    }

    static Optional<String> firstThenLowerCase(
            List<String> strings, Predicate<String> predicate) {
        return strings.stream()
                .filter(predicate)
                .findFirst()
                .map(String::toLowerCase);
    }

    static record Sayer(String phrase) {
        Sayer and(String word) {
            return new Sayer(phrase + ' ' + word);
```

```
        }
    }

    static Sayer say() {
        return new Sayer("");
    }

    static Sayer say(String word) {
        return new Sayer(word);
    }

    static long meaningfulLineCount(String filename) throws IOException {
        try (var reader = new BufferedReader(new FileReader(filename))) {
             return reader.lines()
                .map(String::trim)
                .filter(line -> !line.isBlank() && !line.startsWith("#"))
                .count();
        }
    }
}

record Quaternion(double a, double b, double c, double d) {

    public static final Quaternion ZERO = new Quaternion(0, 0, 0, 0);
    public static final Quaternion I = new Quaternion(0, 1, 0, 0);
    public static final Quaternion J = new Quaternion(0, 0, 1, 0);
    public static final Quaternion K = new Quaternion(0, 0, 0, 1);

    public Quaternion {
        if (Double.isNaN(a) || Double.isNaN(b) || Double.isNaN(c) || Double.isNaN(
            throw new IllegalArgumentException("Coefficients cannot be NaN");
        }
    }

    public Quaternion plus(Quaternion q) {
        return new Quaternion(a + q.a, b + q.b, c + q.c, d + q.d);
    }

    public Quaternion times(Quaternion q) {
        return new Quaternion(
                q.a * a - q.b * b - q.c * c - q.d * d,
                q.a * b + q.b * a - q.c * d + q.d * c,
                q.a * c + q.b * d + q.c * a - q.d * b,
                q.a * d - q.b * c + q.c * b + q.d * a);
    }
```

```
    public Quaternion conjugate() {
        return new Quaternion(a, -b, -c, -d);
    }

    public List<Double> coefficients() {
        return List.of(a, b, c, d);
    }

    @Override public String toString() {
        var builder = new StringBuilder();
        var units = new String[]{"", "i", "j", "k"};
        for (var i = 0; i < units.length; i++) {
            var unit = units[i];
            var c = coefficients().get(i);
            if (c == 0) continue;
            builder.append(c < 0 ? "-" : builder.length() > 0 ? "+" : "");
            builder.append(Math.abs(c) != 1 || unit.isEmpty() ? Math.abs(c) : "");
            builder.append(unit);
        }
        return builder.isEmpty() ? "0" : builder.toString();
    }
}

sealed interface BinarySearchTree permits Empty, Node {
    int size();
    BinarySearchTree insert(String data);
    boolean contains(String data);
}

final record Empty() implements BinarySearchTree {
    @Override public int size() {
        return 0;
    }

    @Override public BinarySearchTree insert(String data) {
        return new Node(data, this, this);
    }

    @Override public boolean contains(String data) {
        return false;
    }

    @Override public String toString() {
        return "()";
    }
}
```

```
final record Node (
        String data, BinarySearchTree left, BinarySearchTree right)
        implements BinarySearchTree {
    @Override public int size() {
        return left.size() + right.size() + 1;
    }

    @Override public BinarySearchTree insert(String data) {
        if (data.compareTo(this.data) < 0) {
            return new Node(this.data, left.insert(data), right);
        } else if (data.compareTo(this.data) > 0) {
            return new Node(this.data, left, right.insert(data));
        } else {
            return this;
        }
    }

    @Override public boolean contains(String data) {
        if (data.compareTo(this.data) < 0) {
            return left.contains(data);
        } else if (data.compareTo(this.data) > 0) {
            return right.contains(data);
        } else {
            return true;
        }
    }

    @Override public String toString() {
        return ("(" + left + data + right + ")").replace("()", "");
    }
}
```

Exercises.kt

```
import java.io.BufferedReader
import java.io.FileReader
import java.io.IOException

fun change(amount: Long): Map<Int, Long> {
    require(amount >= 0) { "Amount cannot be negative" }

    val counts = mutableMapOf<Int, Long>()
    var remaining = amount
    for (denomination in listOf(25, 10, 5, 1)) {
        counts[denomination] = remaining / denomination
```

```
        remaining %= denomination
    }
    return counts
}

fun firstThenLowerCase(strings: List<String>, predicate: (String) -> Boolean): Str
    return strings.firstOrNull(predicate)?.lowercase()
}

data class Sayer(val phrase: String) {
    fun and(word: String): Sayer = Sayer("$phrase $word")
}

fun say(word: String = ""): Sayer = Sayer(word)

@Throws(IOException::class)
fun meaningfulLineCount(filename: String): Long {
    BufferedReader(FileReader(filename)).useLines { lines ->
        return lines.map(String::trim)
            .filter { it.isNotBlank() && !it.startsWith("#") }
            .count()
            .toLong()
    }
}

data class Quaternion(val a: Double, val b: Double, val c: Double, val d: Double)

    companion object {
        val ZERO = Quaternion(0.0, 0.0, 0.0, 0.0)
        val I = Quaternion(0.0, 1.0, 0.0, 0.0)
        val J = Quaternion(0.0, 0.0, 1.0, 0.0)
        val K = Quaternion(0.0, 0.0, 0.0, 1.0)
    }

    init {
        require(!(a.isNaN() || b.isNaN() || c.isNaN() || d.isNaN())) {
            "Coefficients cannot be NaN"
        }
    }

    operator fun plus(q: Quaternion) = Quaternion(a + q.a, b + q.b, c + q.c, d + q

    operator fun times(q: Quaternion) = Quaternion(
        q.a * a - q.b * b - q.c * c - q.d * d,
        q.a * b + q.b * a - q.c * d + q.d * c,
        q.a * c + q.b * d + q.c * a - q.d * b,
```

```
        q.a * d - q.b * c + q.c * b + q.d * a
    )

    fun conjugate() = Quaternion(a, -b, -c, -d)

    fun coefficients() = listOf(a, b, c, d)

    override fun toString(): String {
        val units = listOf("", "i", "j", "k")
        return coefficients().zip(units).fold(StringBuilder()) { sb, (c, unit) ->
            if (c != 0.0) {
                sb.append(if (c < 0) "-" else if (sb.length > 0) "+" else "");
                sb.append(if (Math.abs(c) != 1.0 || unit.isEmpty()) Math.abs(c) el
                sb.append(unit)
            }
            sb
        }.toString().ifEmpty { "0" }
    }
}

sealed interface BinarySearchTree {
    fun size(): Int
    fun insert(data: String): BinarySearchTree
    fun contains(data: String): Boolean

    object Empty : BinarySearchTree {
        override fun size() = 0
        override fun insert(data: String): BinarySearchTree {
            return Node(data, Empty, Empty)
        }
        override fun contains(data: String) = false
        override fun toString() = "()"
    }

    data class Node(
        private val data: String,
        private val left: BinarySearchTree,
        private val right: BinarySearchTree
    ) : BinarySearchTree {
        override fun size() = left.size() + right.size() + 1
        override fun insert(data: String): BinarySearchTree {
            return when {
                data < this.data -> Node(this.data, left.insert(data), right)
                data > this.data -> Node(this.data, left, right.insert(data))
                else -> this
            }
```

```
        }
        override fun contains(data: String): Boolean {
            return when {
                data < this.data -> left.contains(data)
                data > this.data -> right.contains(data)
                else -> true
            }
        }
        override fun toString(): String {
            return ("($left$data$right)").replace("()", "")
        }
    }
}
```

exercises.swift

```
import Foundation

struct NegativeAmountError: Error {}
struct NoSuchFileError: Error {}

func change(_ amount: Int) -> Result<[Int:Int], NegativeAmountError> {
    if amount < 0 {
        return .failure(NegativeAmountError())
    }
    var (counts, remaining) = ([Int:Int](), amount)
    for denomination in [25, 10, 5, 1] {
        (counts[denomination], remaining) =
            remaining.quotientAndRemainder(dividingBy: denomination)
    }
    return .success(counts)
}

func firstThenLowerCase(
    of strings: [String], satisfying predicate: (String) -> Bool
) -> String? {
    return strings.first(where: predicate)?.lowercased()
}

struct Sayer {
    let phrase: String
    func and(_ word: String) -> Sayer {
        return Sayer(phrase: "\(self.phrase) \(word)")
    }
}
```

```
func say(_ word: String = "") -> Sayer {
    return Sayer(phrase: word)
}

func meaningfulLineCount(_ filename: String) async -> Result<Int, Error> {
    var count = 0
    do {
        let url = URL(fileURLWithPath: filename)
        for try await line in url.lines {
            let trimmed = line.trimmingCharacters(in: .whitespaces)
            if !trimmed.isEmpty && !trimmed.hasPrefix("#") {
                count += 1
            }
        }
        return .success(count)
    } catch {
        return .failure(NoSuchFileError())
    }
}

struct Quaternion: CustomStringConvertible, Equatable {
    let a: Double
    let b: Double
    let c: Double
    let d: Double

    static let ZERO = Quaternion()
    static let I = Quaternion(b: 1)
    static let J = Quaternion(c: 1)
    static let K = Quaternion(d: 1)

    init(a: Double = 0, b: Double = 0, c: Double = 0, d: Double = 0) {
        self.a = a
        self.b = b
        self.c = c
        self.d = d
    }

    var coefficients: [Double] {
        return [self.a, self.b, self.c, self.d]
    }

    var conjugate: Quaternion {
        return Quaternion(a: self.a, b: -self.b, c: -self.c, d: -self.d)
    }
```

```
    static func +(q1: Quaternion, q2: Quaternion) -> Quaternion {
        return Quaternion(
            a: q1.a + q2.a,
            b: q1.b + q2.b,
            c: q1.c + q2.c,
            d: q1.d + q2.d
        )
    }

    static func *(q1: Quaternion, q2: Quaternion) -> Quaternion {
        return Quaternion(
            a: q2.a * q1.a - q2.b * q1.b - q2.c * q1.c - q2.d * q1.d,
            b: q2.a * q1.b + q2.b * q1.a - q2.c * q1.d + q2.d * q1.c,
            c: q2.a * q1.c + q2.b * q1.d + q2.c * q1.a - q2.d * q1.b,
            d: q2.a * q1.d - q2.b * q1.c + q2.c * q1.b + q2.d * q1.a
        )
    }

    var description: String {
        var s = ""
        for (c, unit) in zip(self.coefficients, ["", "i", "j", "k"]) where c != 0
            s += c < 0 ? "-" : s == "" ? "" : "+"
            s += abs(c) == 1 && unit != "" ? "" : String(abs(c))
            s += unit
        }
        return s.isEmpty ? "0" : s
    }
}

enum BinarySearchTree: CustomStringConvertible {
    case empty
    indirect case node(String, BinarySearchTree, BinarySearchTree)

    var size: Int {
        switch self {
        case .empty:
            return 0
        case let .node(_, left, right):
            return left.size + right.size + 1
        }
    }

    func insert(_ data: String) -> BinarySearchTree {
        switch self {
        case .empty:
```

```
            return .node(data, .empty, .empty)
        case let .node(selfData, left, right):
            if data < selfData {
                return .node(selfData, left.insert(data), right)
            } else if data > selfData {
                return .node(selfData, left, right.insert(data))
            }
            return self
        }
    }

    func contains(_ data: String) -> Bool {
        switch self {
        case .empty:
            return false
        case let .node(selfData, left, right):
            if data < selfData {
                return left.contains(data)
            } else if data > selfData {
                return right.contains(data)
            }
            return true
        }
    }

    var description: String {
        switch self {
        case .empty:
            return "()"
        case let .node(data, left, right):
            return "(\(left)\(data)\(right))".replacingOccurrences(of: "()", with:
        }
    }
}
```